BIG DATA · Academic year 2026-2027

Assignment 1

Basic Matrix Multiplication in Python, Java and C

Grado en Ciencia e Ingeniería de Datos
Universidad de Las Palmas de Gran Canaria

Carlos Falcón Brito

Repository: https://github.com/Cafabri/Assigment1

Abstract

The naive O(n^3) triple-loop matrix multiplication was implemented in Python 3.11, Java 21 and C (gcc 16.1.0, `-O2`) with the same `ijk` loop order, 64-bit floating point and bit-identical input matrices produced by a common random generator. All implementations were validated against NumPy and benchmarked on an Intel Core i7-13700KF (Windows 11) for $n = 10$ to 3000 under a budget of 15 minutes per run, with five measured runs after warm-up per configuration. Between $n = 100$ and 1000 the runtime grows close to n^3 in the three languages (fitted exponents 3.2–3.3); beyond $n = 1000$, when the matrices no longer fit in the 30 MB L3 cache, the compiled versions grow much faster (exponents 4.6–4.7). C is the fastest at every size. Java needs 1.2–2.1 times the C time up to $n = 1000$ and up to 5.4 times at $n = 1500$; Python needs 130–175 times the C time up to $n = 1500$. C and Java completed every declared size up to $n = 3000$, while Python's largest feasible size was $n = 2000$ (838 s per run). Memory grows as n^2 in all three. These results describe these programs on this computer and should not be generalised to the languages as a whole.

1 Introduction and objective

1.1 Motivation

Matrix multiplication appears in almost every numerical and data-processing workload [1], and its naive form is short enough to be written in exactly the same way in different languages. It is therefore a good test case to see how much the execution environment alone (an interpreter, a JIT compiler or an ahead-of-time compiler) changes the performance of one fixed algorithm. The measurements obtained here are also the baseline for the optimised, parallel and distributed versions of the following assignments.

1.2 Research question

> *How does matrix size affect the runtime and memory usage of a naive triple-loop matrix multiplication implemented in Python, Java and C, when each language runs in its standard execution environment?*

1.3 Variables

- **Factors varied:** the matrix size n (side of the $n \times n$ matrices) and the implementation language.
- **Outcomes measured:** the execution time of the multiplication kernel and the memory used by each implementation.
- **Kept fixed:** algorithm and loop order, numeric type (float64), input matrices, machine, compiler and runtime settings, single thread.

1.4 Hypotheses

The expected behaviour was written down from the theory of Section 2 before running the benchmarks. Table 1 lists each prediction with the result used later to accept or reject it.

Aspect	Prediction	Reason	Checked with
Runtime growth	Time proportional to n^3 in the three languages	The loop performs exactly n^3 multiply-adds	Fig. 1, fitted exponent
Language ranking	C fastest, Java close to C, Python 10–100 times slower than C	Native code vs JIT-compiled code vs interpreter	Table 5 (ratio to C)
Memory growth	Memory proportional to n^2	Matrices have n^2 elements	Table 6 (bytes/n^2)

Table 1: Hypotheses stated before the measurements and how each one is evaluated.

2 Background

2.1 The triple-loop algorithm

Each element of the result is a dot product, $C[i][j] = \sum_k A[i][k] \cdot B[k][j]$. With n^2 elements and n terms per element, the algorithm always performs n^3 multiply-adds, independently of the values, and stores three $n \times n$ matrices [1]. Algorithms with a lower asymptotic cost exist, but this assignment fixes the classical one.

2.2 Why the time per operation is not constant

The n^3 model assumes that every multiply-add costs the same. On real processors, reading data from a cache is much cheaper than reading it from main memory [2]. With the `ijk` order, A is read row by row, but B is read column by column, jumping n elements between consecutive accesses. While the matrices fit in a cache level this is cheap; once they do not, many of these accesses miss the cache and each operation becomes slower [2]. The i7-13700KF has 2 MB of L2 cache per performance core and 30 MB of shared L3.

2.3 How each language executes the loop

- **C** is translated to machine code before execution, so the loop runs directly on the processor.
- **Java** starts executing bytecode and the HotSpot virtual machine compiles the most frequently executed methods to optimised machine code during the run [3]. Early executions are therefore slower, and measurements need warm-up runs and repetitions.
- **CPython** executes the bytecode with an interpreter, and every number is a separate object with its own type information [4]. Each multiply-add therefore involves several interpreted instructions and the creation of a new float object.

3 Methodology

3.1 Implementations and data layout

The three kernels follow the `ijk` order with 64-bit floating-point values (`double` in C and Java, `float` in Python), run on one thread and do not call any numerical library. The way each language stores a matrix cannot be made identical:

- **C:** a single flat array of n^2 doubles, indexed as $i \cdot n + j$ (8 bytes per element, fully contiguous).
- **Java:** `double[][]`, i.e. one array per row. Values are stored as 8-byte doubles, but each row is a separate object, so reading `B[k][j]` requires following a reference to row k first.
- **Python:** a list of lists. The inner lists hold references (8 bytes) to float objects (24 bytes), about 32 bytes per element.

In Java and Python the result matrix is created inside the multiplication function, so its allocation is part of the measured time; in C it is allocated beforehand and only reset to zero, outside the timer. This adds $O(n^2)$ work against the $O(n^3)$ of the loop, and is negligible except for the smallest sizes.

3.2 Input data

To make the three languages multiply exactly the same matrices, the same pseudo-random generator, SplitMix64, was written in C, Java and Python with identical constants. Each 64-bit output is converted to a double in $[0, 10)$ by taking its upper 53 bits, scaling by 2^{-53} and multiplying by 10; matrices are filled row by row, with seed 42 for A and 43 for B. Every operation of this conversion is exact or correctly rounded in IEEE 754, so the generated values are bit-identical in the three languages. This was verified through the checksums described in Section 3.6.

3.3 Environment

Item	Value
CPU	Intel Core i7-13700KF (8 P-cores + 8 E-cores, 24 logical processors)
Cache	2 MB L2 per P-core, 30 MB shared L3
RAM / OS	31.8 GB / Windows 11 Pro
C	gcc 16.1.0 (MSYS2/MinGW64), `-O2`
Java	OpenJDK 21.0.12 (Microsoft Build), default JIT and garbage collector
Python	CPython 3.11.9, standard library only
Analysis	NumPy 2.2 (reference results), pandas, matplotlib

Table 2: Execution environment, identical for all runs (also saved automatically in `results/environment.txt`).

No architecture-specific (`-march=native`) or fast-math flags were used for C. The kernels are single-threaded; the JVM still runs its own JIT-compiler and garbage-collector threads in the background, as in any normal Java execution.

3.4 Measurement protocol

A script (`run_experiment.py`) compiles the C and Java programs and runs every configuration in a separate process, one language after another, so that no two benchmarks run at the same time.

- **Sizes:** $n \in \{10, 50, 100, 200, 300, 500, 750, 1000, 1500, 2000, 3000\}$, declared before the run and identical for the three languages.
- **Budget:** 15 minutes per individual run and 30 GB of memory. If a run exceeds the time limit, the script stops the process, records the configuration as TIMEOUT and does not run larger sizes for that language (SKIPPED). No time is assigned to these configurations.
- **What is timed:** only the call to the multiplication function; input generation happens before the timer starts.

- **Clocks:** all monotonic; C uses the Windows `QueryPerformanceCounter` function, Java uses `System.nanoTime` and Python uses `time.perf_counter`. Times are stored in nanoseconds.
- **Warm-up and repetitions:** 2 (C), 3 (Java) and 1 (Python) warm-up runs, followed by 5 measured runs. Warm-up runs are kept in the raw data but excluded from the statistics. Each configuration is summarised by the median and the interquartile range (IQR).
- **Result consumption:** after every run, each program computes the sum of all elements of the result (checksum) and writes it with the measurement, so the multiplication cannot be skipped by the compiler.

3.5 Memory measurement

There is no single memory tool that works in the three languages, so each one uses its own metric:

- **C:** size of the three allocated arrays, $3 \times 8n^2$ bytes (computed from the allocation, not measured).
- **Java:** increase of the used heap during the call, `totalMemory()` minus `freeMemory()`, read after a `gc()`.
- **Python:** peak traced by `tracemalloc`, started just before the call.

Java and Python therefore see only the memory allocated inside the function, which is the result matrix. Because `tracemalloc` traces every allocation and slows the Python kernel down considerably, Python memory is measured in one additional run that is not used for timing, and only for $n \leq 1000$, where that run still fits in the time budget.

3.6 Correctness

Validation was done in three steps before analysing the timings:

1. **Exact cases.** Each program checks, at start-up, the product by the identity matrix, by the zero matrix, and [[1,2],[3,4]] × [[5,6],[7,8]] = [[19,22],[43,50]], with exact equality.
2. **Reference implementation.** For $n = 50$ and $n = 257$, each program writes its full result matrix, which is compared element by element with NumPy's `A @ B` computed on the same matrices. Since all inputs are non-negative, the sums have no cancellation and their relative rounding error is bounded by about nu, with $u = 2^{-53} \approx 1.1 \times 10^{-16}$ [5]: about 3×10^{-14} for $n = 257$ and 3×10^{-13} for $n = 3000$. The acceptance criterion is a relative error below 10^{-11} for every element, more than 30 times above that bound, while an indexing error would produce relative errors close to 1. No absolute tolerance is needed: every reference element is a sum of n positive products, so none of them is close to zero.
3. **Checksums.** The checksum of every benchmark run is compared across runs, across languages and with the checksum of NumPy's result for the same n.

4 Results

4.1 Correctness

All validation steps passed (Table 3). The three implementations produced bit-identical result matrices, as expected since they execute the same operations in the same order on the same inputs. The checksum was identical in every run of every configuration and in the three languages, so all timed runs computed the correct product.

Test	**Sizes**	**Largest deviation**	**Result**
Identity, zero and 2×2 cases	2	exact	passed (all)
Element-wise vs NumPy	50, 257	rel. error 1.9×10^{-15}	passed (all)
C = Java = Python, element-wise	50, 257	bit-identical	passed
Checksums vs NumPy	all tested	rel. difference 1.2×10^{-13}	passed (all)
Checksums across runs and languages	all tested	bit-identical	passed

Table 3: Correctness checks (tolerance: relative error $\leq 10^{-11}$). Details in `results/correctness.csv` and `results/checksums.csv`.

4.2 Runtime

Table 4 gives the median and IQR of the five measured runs of every configuration, Table 5 the time per multiply-add and the ratio to C, and Figures 1 and 2 show both quantities graphically.

n	**C (s)**	**Java (s)**	**Python (s)**
10	0.0000003 (0.0000000)	0.0000220 (0.0000006)	0.0000522 (0.0000044)
50	0.0000338 (0.0000010)	0.0001015 (0.0001076)	0.005667 (0.000084)
100	0.0003077 (0.0000016)	0.0004646 (0.0000346)	0.04553 (0.00058)
200	0.002734 (0.000042)	0.003250 (0.000024)	0.3747 (0.0019)
300	0.009909 (0.000239)	0.01277 (0.00014)	1.312 (0.006)
500	0.04691 (0.00058)	0.08651 (0.00109)	6.720 (0.002)
750	0.1756 (0.0022)	0.3689 (0.0010)	25.42 (0.08)
1000	0.4729 (0.0021)	0.7520 (0.0050)	72.32 (0.39)
1500	1.936 (0.008)	10.50 (0.05)	317.4 (0.5)
2000	11.10 (0.06)	30.75 (0.06)	838.0 (2.6)
3000	73.81 (0.26)	125.9 (0.5)	TIMEOUT

Table 4: Median kernel time in seconds, with the IQR in parentheses (5 measured runs per configuration, warm-ups excluded).

Variability is low: for every configuration whose median exceeds 1 ms, the IQR is at most 2.4% of the median. Only some of the shortest runs are noisy, in particular Java at $n = 50$ (IQR 106% of the median). A least-squares fit of $\log t$ against $\log n$ gives growth exponents of 3.17 (C), 3.31 (Java) and 3.19 (Python) for $100 \leq n \leq 1000$, and 4.69 (C), 4.60 (Java) and 3.54 (Python) for $n \geq 1000$. When n doubles from 100 to 200, the time grows by a factor of 8.9 (C), 7.0 (Java) and 8.2 (Python), against 8 predicted; from 1000 to 2000 it grows by 23.5 (C), 40.9 (Java) and 11.6 (Python).

n	C (ns)	Java (ns)	Python (ns)	Java / C	Python / C
10	0.30	22.0	52.2	73	174
50	0.27	0.81	45.3	3.0	168
100	0.31	0.46	45.5	1.5	148
200	0.34	0.41	46.8	1.2	137
300	0.37	0.47	48.6	1.3	132
500	0.38	0.69	53.8	1.8	143
750	0.42	0.87	60.3	2.1	145
1000	0.47	0.75	72.3	1.6	153
1500	0.57	3.11	94.0	5.4	164
2000	1.39	3.84	104.7	2.8	75
3000	2.73	4.66	—	1.7	—

Table 5: Time per multiply-add (median time $/n^3$, in nanoseconds) and median runtime relative to C at the same n.

For Java, the first warm-up run was clearly slower than the measured runs at small sizes (13 times the median at $n = 50$ and 6 times at $n = 100$), and within 10% of the median from $n = 500$. For Python, the memory run with `tracemalloc` took 4.7–11 times longer than the median timed run for $n \geq 100$.

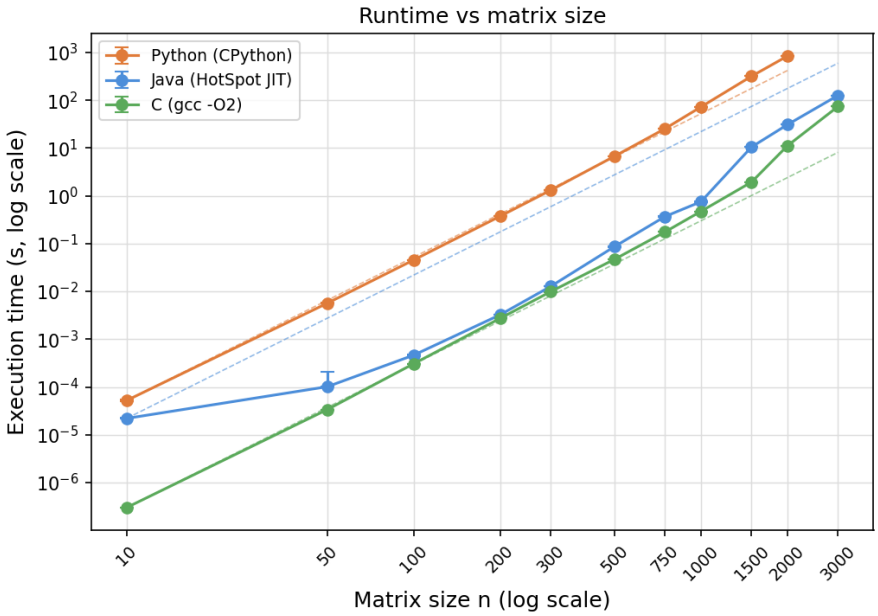


Figure 1: Median runtime vs matrix size n (log-log scale). Dotted lines show the cubic model $t \propto n^3$ anchored at each language's smallest size. Error bars show the IQR (mostly smaller than the markers).

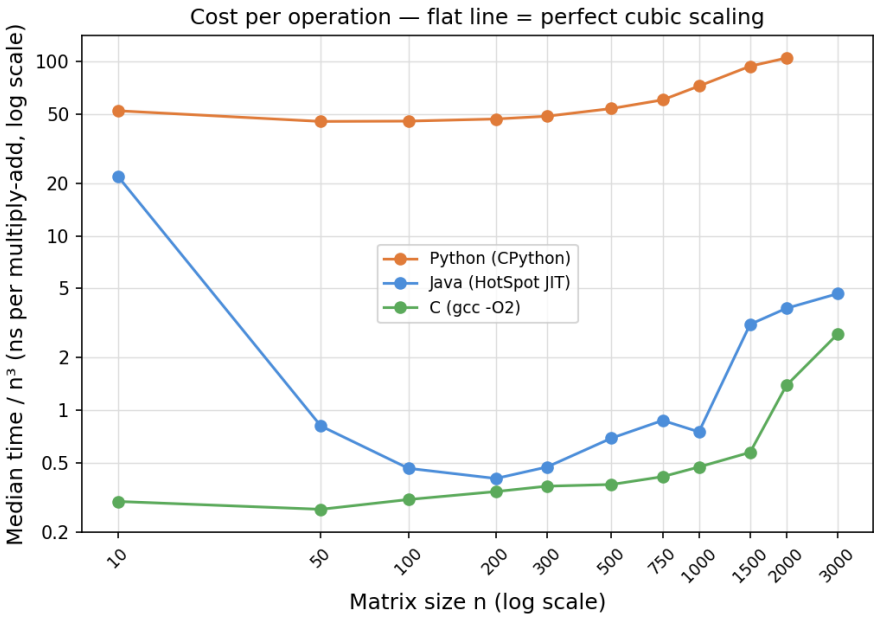


Figure 2: Median time divided by n^3 (nanoseconds per multiply-add). If the runtime were exactly proportional to n^3, each line would be horizontal.

4.3 Memory

Table 6 gives the median memory of each configuration, measured as described in Section 3.5, and Figure 3 shows its growth with n.

n	C (KB)	Java (KB)	Python (KB)
10	2.3	8.7	1.1
100	234.4	92.5	312.4
500	5859.4	2011.1	7837.7
1000	23437.5	8187.8	31307.0
2000	93750.0	31983.5	—
3000	210937.5	71547.1	—
Bytes / n^2	24.0	8.1–8.4	32.0–32.2

Table 6: Median memory (KB = 1024 bytes). C: three matrices. Java and Python: memory allocated during the multiplication (result matrix). Python memory is measured for $n \leq 1000$. The last row gives the range of bytes per element for $n \geq 200$.

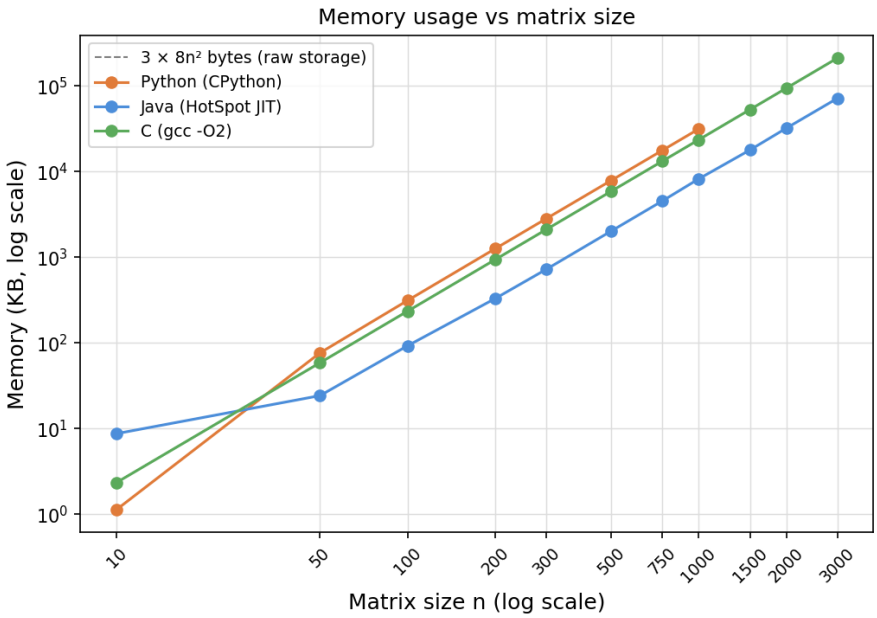


Figure 3: Memory usage vs matrix size n (log-log scale). The dashed line shows the raw storage of three float64 matrices ($3 \times 8n^2$ bytes), which coincides with the C values.

4.4 Largest feasible size

Table 7 shows, for each language, the largest matrix size that finished within the 15-minute budget per run and the first size that exceeded it.

Language	**Largest feasible n**	**Longest run**	**First size over budget**
C	3000	74 s ($n = 3000$)	none
Java	3000	126 s ($n = 3000$)	none
Python	2000	841 s ($n = 2000$)	3000 (TIMEOUT at 900 s)

Table 7: Limits under the 15-minute-per-run budget (`results/run_status.csv`). The largest size attempted was $n = 3000$ for all three languages. No configuration came close to the memory budget: the largest value was 206 MiB (C, $n = 3000$).

5 Discussion

5.1 Evaluation of the hypotheses

In answer to the research question: memory grows as n^2 in the three languages, and runtime grows as n^3 only while the matrices fit in the cache; beyond that size the time per operation increases and the growth is steeper. The language changes the runtime by a large factor (Python 75–175 times slower than C), but this factor is not fixed: the Java–C gap in particular depends on n.

Table 8 compares each prediction of Table 1 with the measurements; the following subsections explain the deviations.

Hypothesis	**Observed**	**Verdict**
Runtime $\propto n^3$	Exponents 3.17–3.31 for $100 \leq n \leq 1000$; from $n = 1500$ C and Java grow much faster (exponents 4.6–4.7)	Partly confirmed (only up to $n = 1000$)
Language ranking	Order C < Java < Python at every size. Java 1.2–2.1 times C up to $n = 1000$, but 5.4 at $n = 1500$. Python 130–175 times C, above the predicted 10–100	Partly confirmed (order correct, gaps not)
Memory $\propto n^2$	Bytes/n^2 constant in each language for $n \geq 200$ (24.0, 8.1–8.4 and 32.0–32.2 B)	Confirmed

Table 8: Verdict for each hypothesis of Table 1.

5.2 Growth with matrix size

The cubic prediction of Table 1 holds only while the matrices fit in the cache. For $100 \leq n \leq 1000$ the three implementations grow with exponents between 3.17 and 3.31, and doubling n multiplies the time by 7.0–8.9, close to the factor of 8 of the model. Beyond $n = 1000$ the compiled versions depart strongly from it: from $n = 1000$ to 2000 the C time grows 23.5 times and the Java time 40.9 times. The number of operations still follows n^3 exactly, so this extra growth comes from each operation becoming more expensive, which the n^3 model does not capture.

The point where this happens is consistent with the cache sizes. Three matrices of doubles occupy $24n^2$ bytes: 24 MB at $n = 1000$, which still fits in the 30 MB L3 cache, and 54 MB at $n = 1500$, which does not. In C the cost per operation stays between 0.27 and 0.47 ns up to $n = 1000$ and then rises to 0.57, 1.39 and 2.73 ns at $n = 1500$, 2000 and 3000 (Figure 2). At these sizes, consecutive accesses to a column of B are $8n$ bytes apart (16 KB at $n = 2000$), so almost every access needs a new cache line from main memory [2]. This explanation agrees with the measurements, but cache misses were not measured with hardware counters, so it remains a hypothesis.

At small sizes the cubic model does not describe Java either: at $n = 10$ one multiply-add costs 22 ns, and the first warm-up run at $n = 100$ was six times slower than the measured median. Both are consistent with fixed overheads and with code that the JIT compiler has not yet optimised [3].

5.3 Differences between languages

The predicted ranking is correct at every size: C is the fastest, followed by Java and then Python. The size of the gaps, however, depends on n, so no single factor describes them.

Java vs C. For $100 \leq n \leq 1000$ Java needs 1.2–2.1 times the C time: once the loop is compiled, both execute native code, as predicted. At $n = 1500$ the gap jumps to 5.4, because the cost per operation of Java grows from 0.75 to 3.11 ns while that of C grows only from 0.47 to 0.57 ns. Both store 8-byte doubles, so the amount of data is the same; the most likely difference is how B is stored. In C, the address of $B[k][j]$ is computed directly from k and j inside one flat array. In Java, every step of the inner loop first reads the reference of row k from the outer array and only then the value, so each operation needs one extra memory access, and the row arrays may be spread over the heap [2]. For larger sizes the gap narrows again (1.7 at $n = 3000$), because C also becomes limited by main memory. The experiment cannot separate the layout from other JIT effects such as bounds checks; this question is left for Assignment 2. The small drop of the Java cost per operation at $n = 1000$ (0.75 ns against 0.87 ns at $n = 750$) is not explained by these data.

Python vs C. Python needs 130–175 times the C time up to $n = 1500$. This is above the predicted range of 10–100, so the prediction is only partly confirmed: the ranking is right, but the interpreter overhead was underestimated. CPython spends about 45 ns per multiply-add even for small matrices, against 0.3 ns in C, because each operation goes through the interpreter, checks the types of its operands and creates a new float object [4]. This fixed cost dominates at every size; Python's cost per operation increases only gradually with n (45 to 105 ns), and the Python/C ratio drops to 75 at $n = 2000$ only because C slows down, not because Python gets faster.

5.4 Memory

The quadratic growth is confirmed: the memory divided by n^2 is practically constant in each language (24.0 B in C, 8.1–8.4 B in Java and 32.0–32.2 B in Python for $n \geq 200$). The absolute values are not comparable, because C counts three matrices and Java and Python only the result. Comparing the same object, the result matrix, Python needs 32 bytes per element against 8 in Java and C, i.e. four times more, because of the reference plus the float object. The Java values for $n \leq 50$ are dominated by allocation buffers and the garbage collector and are not meaningful. The largest memory value, 206 MiB, is far below the 30 GB budget; the only limit reached in the experiment was the time limit.

5.5 Limitations

- The study uses a single computer, operating system and set of language versions; other hardware, in particular other cache sizes, would move the points where the cost per operation increases.
- Five repetitions per configuration give only a rough estimate of variability, and runs shorter than about 0.1 ms are only a few ticks of the clock.
- The memory metric is different in each language, approximate in Java because of garbage collection, and available for Python only up to $n = 1000$.

- The explanations based on the cache and on the Java row layout are inferred from the timings only.
- The declared size sequence ended at $n = 3000$, where C and Java still ran far below the budget (74 s and 126 s per run). Their actual largest feasible size is therefore larger and was not determined.

6 Conclusions

For the three implementations and the conditions described in Section 3:

- Up to $n = 1000$ the runtime agrees with the n^3 model (exponents 3.2–3.3). From $n = 1500$, once the three matrices exceed the L3 cache, C and Java become much slower than the model predicts (exponents 4.6–4.7), so the cost of an operation depends on the matrix size and not only on the number of operations.
- C is the fastest at every size. Java needs 1.2–2.1 times the C time up to $n = 1000$, up to 5.4 times at $n = 1500$, and 1.7 times at $n = 3000$. Python needs 130–175 times the C time up to $n = 1500$, more than predicted.
- Memory grows as n^2; for the same result matrix Python uses four times more memory than Java or C.
- Under the 15-minute budget, C and Java completed every declared size up to $n = 3000$, and Python up to $n = 2000$.

These results compare specific implementations on one machine; they are not a general ranking of programming languages. Two questions remain open: whether the jump in the cost per operation is caused by cache and memory-access effects, and how much of the Java–C gap comes from the `double[][]` layout. Both are natural starting points for the loop-reordering and blocking optimisations [2] of Assignment 2.

Reproducibility materials

All materials are in the repository linked on the first page: the source code, the build and run commands (`README.md`), the raw measurements of every run (`results/raw_measurements.csv`), the status of every configuration (`results/run_status.csv`), the correctness results and the analysis scripts. The whole study is reproduced with `run_experiment.py`, `analysis/check_correctness.py` and `analysis/plot_results.py`.

Use of generative AI

Claude Sonnet 4.6 and Claude Opus 5.5 (Anthropic), used through Claude Code, served as support tools during this assignment: to help write and debug parts of the benchmark and analysis scripts, and to improve the structure and wording of the report. All the measurements come from running the code on the machine described above. The author reviewed the code and the AI-generated text, checked the numbers against the raw data and the references, and is responsible for the conclusions.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[2] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2019.

[3] Oracle, "Java HotSpot virtual machine performance enhancements," in *Java Virtual Machine Guide, Release 21*. [Online]. Available: https://docs.oracle.com/en/java/javase/21/vm/java-hotspot-virtual-machine-performance-enhancements.html (accessed Oct. 15, 2026).

[4] Python Software Foundation, "Data model," in *The Python Language Reference*, Python 3.11 documentation. [Online]. Available: https://docs.python.org/3.11/reference/datamodel.html (accessed Oct. 14, 2026).

[5] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Comput. Surv.*, vol. 23, no. 1, pp. 5–48, Mar. 1991, doi: 10.1145/103162.103163.